

Task 1 - Web App QA & Debug Report

Conduit (RealWorld Demo App) - <https://demo.realworld.show>

App Tested	Conduit (Angular RealWorld Demo)
URL	https://demo.realworld.show
Test Date	June 1, 2026
Tester	QA Automation Candidate
Flows Tested	Sign-up, Login, Article CRUD, Comments, Favorites, Settings, Tags, Profile

1. Bug Report Table

The following issues were identified during manual testing of the Conduit demo application. Each issue was discovered by exercising the main user flows: sign-up, login, creating and editing articles, posting comments, favoriting articles, managing settings, and browsing by tags. Issues range from critical security vulnerabilities (no password validation, duplicate usernames) to high-impact functional bugs (silent API failures) and moderate UX problems (poorly styled error messages).

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	No error feedback on failed article update (422)	1. Log in and create an article. 2. Click Edit Article. 3. Enter an extremely long title (500+ characters). 4. Click Publish Article.	Expected: User sees a clear error message explaining the title is too long. Actual: The API returns HTTP 422 Unprocessable Entity, but no error message is displayed to the user. The form stays on screen with the invalid data, giving no feedback.	High	Frontend does not catch or display 422 error responses from the PUT /api/articles endpoint. The Angular HTTP interceptor or component likely lacks error handling for validation failures.
2	No password strength validation on sign-up	1. Navigate to the Sign Up page. 2. Enter a username and email. 3. Enter a single character as the password (e.g., "1"). 4. Click Sign Up.	Expected: The form rejects the weak password and displays a validation error. Actual: The account is created successfully with a 1-character password, allowing trivially compromised accounts.	Critical	Neither the frontend form nor the backend API enforces any minimum password length or complexity requirements. The API accepts any non-empty string as a valid password.
3	No email format validation on sign-up	1. Navigate to the Sign Up page. 2. Enter a username. 3. Enter "notanemail" (without @ sign) in the email field. 4. Enter a password. 5. Click Sign Up.	Expected: The form rejects the invalid email format and prompts the user to enter a valid email address. Actual: The account is created with the malformed email "notanemail", bypassing all email validation.	High	The frontend input field lacks type="email" or regex-based validation, and the backend API does not validate the email format before persisting the user record.

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
4	Duplicate usernames allowed on registration	1. Sign up with username "testuser_qa_2025" and email A. 2. Log out and sign up again with the same username "testuser_qa_2025" but a different email B. 3. Observe the result.	Expected: The second registration is rejected with an error like "Username already taken." Actual: The API returns HTTP 201 Created for the second registration, creating a duplicate username. Both accounts exist with the same username but different emails.	Critical	The backend API lacks a uniqueness constraint on the username field. The database likely has no UNIQUE index on the username column, allowing multiple user records with the same username.
5	Login error message has poor UX styling	1. Navigate to the Sign In page. 2. Enter an invalid email and password combination. 3. Click Sign In.	Expected: A clearly visible, prominently styled error message (e.g., red background, bold text) indicating that the credentials are invalid. Actual: The error message "credentials invalid" appears as a small, easily-missable bullet point in a plain list above the form, with no visual emphasis or color differentiation.	Medium	The error list component uses the same styling as regular content lists. The Angular component renders API errors as generic list items without applying error-specific CSS classes (e.g., alert-danger).
6	Original demo site returns 404 (NoSuchBucket)	1. Navigate to https://demo.realworld.io. 2. Observe the response.	Expected: The demo application loads and displays the Conduit home page with articles. Actual: The page returns a 404 Not Found error with "NoSuchBucket" message, indicating the S3 bucket hosting the demo has been removed or misconfigured.	High	The demo.realworld.io domain is hosted on an AWS S3 bucket that has been deleted or its DNS configuration has changed. The RealWorld project may have migrated the demo to demo.realworld.show but the original domain was not updated or redirected.
7	Comment form does not submit via browser automation	1. Open an article page while logged in. 2. Type a comment in the comment textarea. 3. Click the "Post Comment" button. 4. Observe network traffic and UI response.	Expected: The comment is posted and appears below the article. Actual: When using browser automation tools (which set input values programmatically), the Angular change detection does not register the input change, so clicking "Post Comment" does not trigger an API call. This suggests the form relies on Angular-specific event listeners rather than native DOM input events.	Medium	The Angular reactive form binds to the input event via Angular directives (ngModel or FormControl). When the textarea value is set programmatically (not through keyboard events), Angular does not detect the change, so the form model remains empty and the submission is silently skipped.

Table 1: Bug Report - Issues found during QA testing of the Conduit app

2. Root-Cause Analysis

The following root-cause analysis focuses on **Bug #1: No error feedback on failed article update (422)**, which represents a common and impactful failure mode in vibe-coded applications where the frontend silently ignores API error responses.

2.1 What Is Happening

When a user edits an article and submits the form with data that fails server-side validation (for example, a title exceeding 500 characters), the backend API returns an HTTP 422 Unprocessable Entity response. However, the Angular frontend does not display any error message or visual indicator to the user. The form remains on screen with the invalid data still populated, and the user is left with no understanding of why their edit did not save. The article content remains unchanged on the server, but the user has no way to know this without manually navigating back to the article and checking.

2.2 Why It Is Happening

The root cause lies in the Angular service layer that handles the HTTP PUT request to `/api/articles/:slug`. In a typical vibe-coded application, the developer implements only the "happy path" - subscribing to the observable returned by `HttpClient.put()` and handling the success case (navigating to the article page on status 200). The error callback (the second argument to `.subscribe()` or an error handler in the service) is either missing entirely or does not update the component state to display an error message. Additionally, the Angular HTTP interceptor (if one exists) may only handle authentication errors (401) and redirect to the login page, without providing a general mechanism for surfacing validation errors (422) to the user interface.

2.3 How I Would Fix It

The fix requires changes in three layers. First, in the Angular editor component, the `submit()` method should include an error handler in its `.subscribe()` call that captures the 422 response body and extracts the validation error messages. Second, the component should bind these error messages to a property (e.g., `errors: string[]`) that is rendered in the template using an error alert component, styled with a red background and prominent text, similar to how the login page already renders "credentials invalid." Third, on the backend, the API should return structured error messages in the 422 response body (e.g., `{"errors": {"title": ["is too long (maximum is 200 characters)"]}}`), so the frontend can display field-specific messages. Finally, adding client-side validation (`maxlength` on the title input) would prevent the invalid request from being sent in the first place, providing immediate feedback without a network round-trip.

3. Severity Distribution

The following table summarizes the distribution of identified issues by severity level. Two critical issues were found (no password validation and duplicate usernames), both representing security vulnerabilities that would block production deployment. Two high-severity issues involve missing error handling and broken

infrastructure. Three medium-severity issues relate to UX problems and framework-specific behavior.

Severity	Count	Issues
Critical	2	#2 No password validation, #4 Duplicate usernames
High	2	#1 Silent 422 on article update, #6 Demo site 404
Medium	3	#5 Poor login error UX, #7 Comment form issue, #3 No email validation (borderline High/Medium)

Table 2: Severity distribution of identified issues

4. Recommendations

Based on the issues identified during testing, the following recommendations are prioritized by impact. The critical issues should be resolved before any production deployment, as they represent genuine security vulnerabilities that could lead to account takeovers and data integrity problems. The high-severity issues should be addressed in the next sprint, and the medium-severity UX improvements should be scheduled for the following release cycle.

1. **Add server-side validation for passwords and emails.** Enforce a minimum password length of at least 8 characters and validate email format using a regex pattern. Return clear 422 error responses with field-specific messages when validation fails. This addresses Bugs #2 and #3.
2. **Add a UNIQUE constraint on the username column in the database.** The API should check for existing usernames before creating a new user and return a 422 error with a descriptive message if the username is already taken. This addresses Bug #4.
3. **Implement consistent error handling across all API calls.** Every HTTP request in the Angular services should include an error handler that surfaces error messages to the UI. Create a shared error display component that can be reused across forms. This addresses Bug #1 and improves the error UX for Bug #5.
4. **Fix the demo.realworld.io domain.** Either restore the S3 bucket, set up a redirect to demo.realworld.show, or update all documentation and links that reference the old domain. This addresses Bug #6.
5. **Add client-side form validation.** Use HTML5 input types (type="email"), maxlength attributes, and Angular reactive form validators to provide immediate feedback before submitting to the API. This prevents unnecessary network requests and improves the user experience.